

## **1. Introduction**

L'objectif de ce projet est de créer un petit jeu très simple avec une fenêtre à l'écran dans laquelle un cercle est affiché. Ce cercle représente un objet qui peut monter et descendre. Le but principal n'est pas de créer un jeu compliqué, mais de comprendre comment un programme peut afficher quelque chose à l'écran, réagir aux actions de l'utilisateur et fonctionner de manière fluide.

Dans cette application, le cercle descend tout seul au fil du temps. Lorsque l'utilisateur clique avec la souris dans la fenêtre, le cercle monte vers le haut, comme s'il sautait. Cela permet de montrer comment un programme peut réagir aux actions de l'utilisateur et mettre à jour l'affichage en conséquence.

Le projet a aussi pour objectif d'apprendre à organiser correctement le travail. Chaque partie du programme a un rôle bien précis : une partie s'occupe de mémoriser les informations importantes, une autre s'occupe de l'affichage, et une autre encore gère les actions de l'utilisateur. Cette organisation rend le programme plus clair et plus facile à comprendre.

Ce travail constitue une première étape avant des projets plus complets. Il permet de poser les bases nécessaires pour ajouter plus tard de nouvelles fonctionnalités, comme un parcours à suivre ou des déplacements plus complexes.

## **2. Analyse globale**

### **Analyse globale des fonctionnalités**

Le projet a pour objectif de développer une application graphique interactive en Java, structurée selon le patron de conception MVC. Afin d'atteindre cet objectif, plusieurs fonctionnalités principales ont été identifiées. Ces fonctionnalités constituent la base du projet et seront détaillées et planifiées dans la section suivante (plan de développement).

Les fonctionnalités principales du projet sont les suivantes :

1. Affichage d'une interface graphique
2. Interaction utilisateur par clic de souris
3. Gestion du mouvement vertical du cercle
4. Mise à jour automatique de l'affichage
5. Génération et affichage d'un parcours sous forme de ligne brisée
6. Organisation du code selon le modèle MVC

## **Analyse détaillée des fonctionnalités**

### **1. Affichage de l'interface graphique**

Cette fonctionnalité consiste à créer une fenêtre graphique et à y afficher les éléments visuels du projet, en particulier le cercle représentant l'objet principal.

- **Difficulté** : faible
- **Priorité** : très élevée

Cette fonctionnalité est indispensable, car aucune autre fonctionnalité ne peut être utilisée sans interface graphique visible.

### **2. Interaction utilisateur par clic de souris**

Lorsqu'un clic est détecté, une action est déclenchée afin de modifier l'état du modèle.

- **Difficulté** : moyenne
- **Priorité** : très élevée

Elle est essentielle pour rendre l'application interactive et permet de faire le lien entre l'utilisateur et le comportement du cercle.

### **3. Gestion du mouvement vertical du cercle**

Le cercle doit monter lorsqu'un clic est effectué et redescendre progressivement lorsqu'aucune interaction n'a lieu. Cette fonctionnalité repose sur la modification de la position verticale du cercle stockée dans le modèle.

- **Difficulté** : moyenne
- **Priorité** : élevée

### **4. Mise à jour automatique de l'affichage**

Afin de garantir une animation fluide, l'affichage doit être rafraîchi régulièrement. Cette fonctionnalité est assurée par un thread dédié qui redessine l'interface à intervalles réguliers.

- **Difficulté** : moyenne
- **Priorité** : élevée

## 5. Génération et affichage d'un parcours sous forme de ligne brisée

Cette fonctionnalité consiste à générer une ligne brisée à partir d'une liste de points calculés automatiquement, en respectant des contraintes de position. Cette ligne est ensuite affichée dans la vue.

- **Difficulté** : élevée
- **Priorité** : moyenne

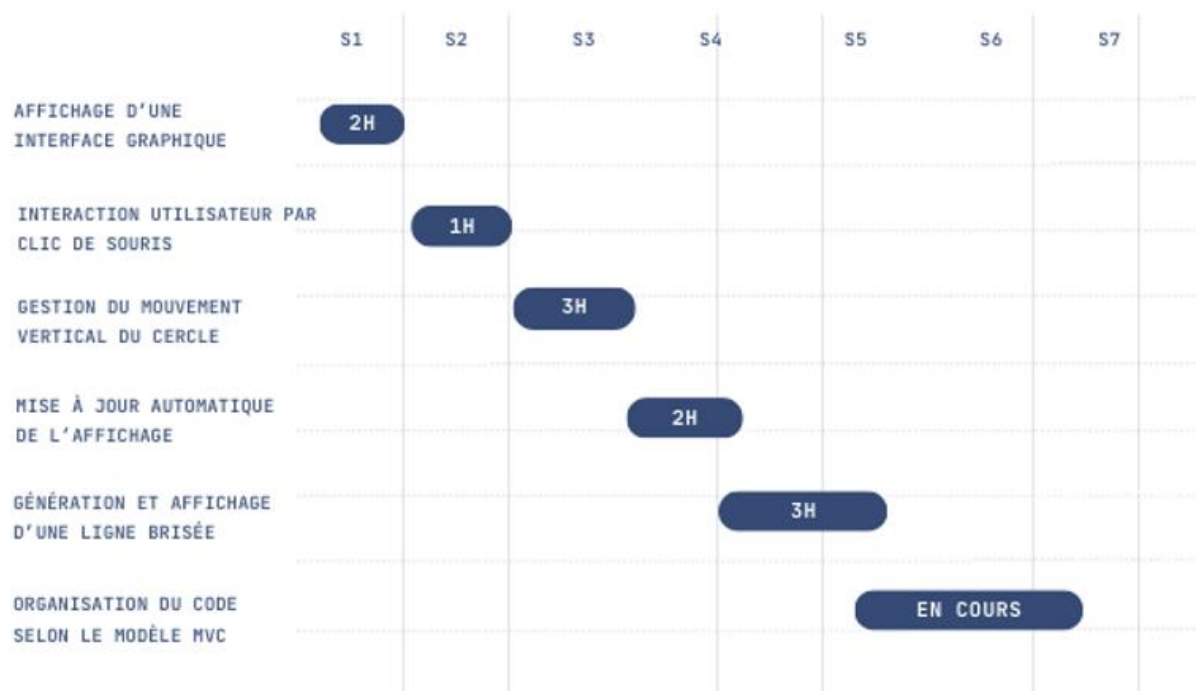
## 6. Organisation du code selon le modèle MVC

Le projet est structuré selon le patron de conception MVC afin de séparer clairement les responsabilités :

- Le modèle gère les données et la logique,
- La vue gère l'affichage,
- Le contrôleur gère les interactions utilisateur.
- **Difficulté** : moyenne
- **Priorité** : très élevée

Cette organisation améliore la lisibilité, la maintenabilité et l'évolutivité du projet.

## 3. Plan de développement



## 4. Conception générale

### Présentation globale de l'architecture

Le projet est organisé selon une architecture en **blocs fonctionnels**, inspirée du modèle MVC (Modèle – Vue – Contrôleur).

Cette organisation permet de séparer clairement les rôles des différentes parties du programme, afin de faciliter la compréhension, la maintenance et l'évolution du projet.

### Description des blocs fonctionnels

#### 1. Bloc Modèle :

Ce bloc représente **l'état du jeu** et son évolution dans le temps.

Il contient toutes les informations nécessaires pour décrire ce qui se passe dans le jeu, indépendamment de l'affichage.

Il gère notamment :

- La hauteur du cercle,
- Les limites minimales et maximales du mouvement,
- La génération de la ligne brisée (parcours),
- L'évolution automatique de la position du cercle (descente).

#### 2. Bloc Vue :

Ce bloc est chargé de **représenter visuellement** l'état du modèle.

Il ne prend aucune décision sur le fonctionnement du jeu : il se contente d'afficher ce que le modèle contient.

Il assure :

- L'affichage du cercle,
- L'affichage de la ligne brisée,
- L'adaptation des coordonnées du modèle vers la fenêtre graphique,
- Le rafraîchissement régulier de l'écran.

### **3. Bloc Contrôleur :**

Ce bloc fait le lien entre l'utilisateur et le modèle.

Il interprète les actions de l'utilisateur et déclenche les modifications nécessaires dans le modèle.

Dans ce projet, le contrôleur :

- détecte les clics de souris,
- déclenche un saut du cercle lorsque l'utilisateur clique.

Le contrôleur ne s'occupe pas de l'affichage : il agit uniquement sur les données du modèle.

### **4. Bloc de gestion du temps (threads)**

Certains comportements du jeu doivent se produire automatiquement, sans action directe de l'utilisateur.

Pour cela, des blocs de gestion du temps sont utilisés.

Ils permettent :

- De faire redescendre le cercle progressivement,
- De rafraîchir l'affichage à intervalles réguliers.

Ces blocs fonctionnent en parallèle et garantissent une animation fluide et continue.

## **5. Conception détaillée**

### **1. Gestion de la position verticale du cercle**

#### **Objectif**

Cette fonctionnalité permet de gérer la position verticale du cercle dans l'application. Elle définit le comportement du cercle lors d'un saut et lors de la descente automatique, tout en garantissant que le cercle reste dans une zone valide de l'écran.

#### **Structures de données et constantes**

La position verticale du cercle est représentée par une valeur numérique correspondant à sa hauteur.

Plusieurs constantes sont utilisées afin d'encadrer ce comportement :

- Une valeur définissant l'intensité du saut,
- Une hauteur minimale autorisée,
- Une hauteur maximale autorisée,

- Une valeur représentant la taille verticale du cercle.

Ces constantes permettent d'assurer un comportement cohérent et contrôlé du cercle.

### **Algorithme (description)**

Lorsqu'un saut est déclenché, la hauteur du cercle est augmentée d'une valeur fixe. Une vérification est ensuite effectuée afin de s'assurer que la hauteur ne dépasse pas la limite maximale autorisée.

En parallèle, une descente automatique est appliquée de manière régulière. À chaque mise à jour, la hauteur du cercle est diminuée progressivement tant que la limite minimale n'est pas atteinte.

### **Conditions limites**

- La hauteur du cercle ne doit jamais dépasser la valeur maximale définie.
- La hauteur ne doit jamais être inférieure à la valeur minimale.
- Le cercle doit rester visible à tout moment.

### **Interaction avec les autres fonctionnalités**

Cette fonctionnalité est utilisée par :

- La vue, qui s'appuie sur la hauteur pour afficher correctement le cercle,
- Le contrôleur, qui modifie la hauteur lors d'un clic utilisateur,
- Le mécanisme de descente automatique, qui agit directement sur la hauteur.

## **2. Interaction utilisateur par clic souris**

### **Objectif**

Cette fonctionnalité permet à l'utilisateur d'interagir avec l'application en déclenchant un saut du cercle à chaque clic dans la fenêtre graphique.

### **Algorithme (description)**

Lorsqu'un clic de souris est détecté :

1. L'événement est capté par le système d'interaction.
2. Une action de saut est déclenchée.
3. La hauteur du cercle est modifiée en conséquence.

La mise à jour visuelle est ensuite prise en charge automatiquement par le rafraîchissement de l'affichage.

## **Interaction avec les autres fonctionnalités**

Cette fonctionnalité modifie l'état du modèle et entraîne une mise à jour indirecte de l'affichage.

### **3. Génération de la ligne brisée (parcours)**

#### **Objectif**

Cette fonctionnalité permet de générer une ligne brisée représentant le parcours du jeu. La ligne doit être continue, varier de manière aléatoire et couvrir toute la zone visible de l'écran.

#### **Structures de données et constantes**

Le parcours est stocké sous la forme d'une liste de points.

Des constantes définissent :

- La distance minimale et maximale entre deux points successifs,
- Les limites verticales autorisées pour les points.

#### **Algorithme (description)**

La génération du parcours se déroule comme suit :

1. Deux premiers points sont créés avec la même hauteur afin de démarrer sur une ligne horizontale.
2. Une position horizontale initiale est définie avant la zone visible.
3. Tant que la fin de la zone n'est pas atteinte :
  - Une distance horizontale aléatoire est ajoutée,
  - Une hauteur aléatoire comprise dans les limites autorisées est choisie,
  - Un nouveau point est ajouté à la liste.
4. Le processus se répète jusqu'à ce que la ligne dépasse légèrement la zone visible.

## **Interaction avec les autres fonctionnalités**

La vue utilise cette liste de points pour dessiner la ligne brisée à l'écran.

## **4. Affichage graphique**

### **Objectif**

Cette fonctionnalité assure la représentation visuelle du cercle et du parcours dans une fenêtre graphique.

### **Structures de données utilisées**

- La hauteur du cercle issue du modèle.
- La liste des points représentant le parcours.

### **Algorithme (description)**

À chaque mise à jour de l’affichage :

1. La zone graphique est nettoyée.
2. Les points du parcours sont parcourus deux par deux.
3. Chaque paire de points est reliée par un segment.
4. La position verticale du cercle est calculée à partir de sa hauteur.
5. Le cercle est dessiné à une position horizontale fixe.

### **Interaction avec les autres fonctionnalités**

L’affichage dépend entièrement des données fournies par le modèle et est rafraîchi automatiquement.

## **5. Mise à jour automatique et animation**

### **Objectif**

Cette fonctionnalité permet d’assurer une animation continue du jeu sans intervention permanente de l’utilisateur.

### **Algorithme (description)**

De manière répétée :

1. Un court délai est respecté.
2. La position du cercle est mise à jour (descente).
3. L’affichage est rafraîchi.
4. Le cycle recommence.



